

# SYSTEMARCHITEKTUR & PFLICHTENHEFT

Projekt: Modulares Unternehmens-Kalendersystem | Bellmann Engineering

## 1. Executive Summary & Zielsetzung

Das vorliegende Dokument beschreibt die vollständige technische Architektur und die funktionalen Anforderungen für die Neuentwicklung des internen Kalendersystems der **Bellmann Engineering**. Das System löst den bisherigen Einsatz von Google Calendar ab, um Terminüberschneidungen, ungewollte Manipulationen durch Mitarbeiter, Datenschutzrisiken und den Verlust administrativer Kontrolle endgültig zu eliminieren.

### Kernergebnisse & Hauptziele:

- **Strikte Trennung von Zuständigkeiten (SoC):** Reines Leserecht für Trainer; Bearbeitungsrechte nur für Team Leader und den CEO.
- **Umfassende Modularisierung:** Vollständig entkoppelte Dateistruktur nach Domain-Driven-Design zur Vermeidung von 'Monolith-Code'.
- **KI-Konfliktanalyse im Casino-Card-Layout:** Nebeneinanderliegende Vergleichskarten statt störender Kalender-Überlagerungen.
- **Lückenlose Transparenz:** Unveränderbares Audit-Log für jede Aktion im System.

## 2. Modulare Architektur & Dateistruktur

Um Wartbarkeit, Testbarkeit und Erweiterbarkeit zu garantieren, wird der Quellcode in spezialisierte Einzelmodule unterteilt. **Grundsatz:** Keine Datei darf mehr als eine klar definierte Verantwortung besitzen (Single Responsibility Principle). Nur bei zwingender technischer Abhängigkeit (z. B. eng gekoppelte Models/Enums) werden Klassen im selben Modul zusammengefasst.

| Ordner / Datei    | Funktion & Verantwortung                                                                                                                                                                                 | Begründung der Entkopplung                                                                                                           |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| `app/__init__.py` | Application Factory (`create_app`)                                                                                                                                                                       | Verhindert zirkuläre Importe und ermöglicht saubere Test-Umgebungen.                                                                 |
| `app/config.py`   | Zentrale Konfigurationsklassen                                                                                                                                                                           | Isoliert Umgebungsvariablen (Dev, Staging, Prod) von der Anwendungslogik.                                                            |
| `app/models/`     | SQLAlchemy Datenbank-Modelle <ul style="list-style-type: none"><li>- `user.py`</li><li>- `event.py`</li><li>- `skill.py`</li><li>- `audit.py`</li></ul>                                                  | Jede Datenbank-Entität erhält eine eigene Datei. Verhindert unübersichtliche Riesendateien und erleichtert Schema-Migrationsskripte. |
| `app/routes/`     | Flask Blueprints (HTTP Controllers) <ul style="list-style-type: none"><li>- `auth_routes.py`</li><li>- `calendar_routes.py`</li><li>- `admin_routes.py`</li><li>- `ai_routes.py`</li></ul>               | Routendefinitionen enthalten KEINE Geschäftslogik. Sie validieren lediglich Requests und delegieren an die Service-Schicht.          |
| `app/services/`   | Reine Geschäftslogik (Pure Python) <ul style="list-style-type: none"><li>- `calendar_service.py`</li><li>- `conflict_service.py`</li><li>- `ai_service.py`</li><li>- `notification_service.py`</li></ul> | Geschäftsregeln (Pufferzeiten, Kollisionsprüfungen) bleiben unabhängig von HTTP-Routen testbar.                                      |
| `app/decorators/` | Sicherheits- & Rechte-Dekoratoren <ul style="list-style-type: none"><li>- `auth_decorators.py`</li></ul>                                                                                                 | Zugriffskontrollen (`@role_required`, `@ceo_override`) werden zentral definiert und wiederverwendet.                                 |
| `prompts/`        | `conflict_prompt.txt`                                                                                                                                                                                    | KI-Prompts werden strikt als Textdateien ausserhalb des Python-Codes gepflegt.                                                       |

## 3. Best Practices & Entwicklungsstandards

| Best Practice               | Technische Umsetzung                                                                              | Warum essenziell? (Begründung)                                                                     |
|-----------------------------|---------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Single Responsibility (SRP) | Strikte Trennung in Routes, Services, Models und Decorators.                                      | Verhindert Monolithen. Änderungen an der KI-Logik beeinflussen nicht die Datenbank-Routen.         |
| Type Hints & Static Checks  | Konsequente Nutzung von Python Type Hints ( <code>`str`, `int`, `Optional[List[Event]]`</code> ). | Erkennt Typenfehler bereits während der Entwicklung vor der Laufzeit (via Static Analysis / IDE).  |
| Security by Design (RBAC)   | Prüfung der Zugriffsrechte im Backend-Decorator VOR dem Datenbankzugriff.                         | Schützt vor Manipulation von HTTP-Parametern im Frontend durch Unbefugte.                          |
| Soft-Delete Pattern         | Feld <code>`is_deleted = Column(Boolean, default=False)`</code> in den Modellen.                  | Bewahrt die Historie für Audit-Logs und KI-Berichte. Hard Delete ist nur für Kai/Admins verfügbar. |
| Zentrale Fehlerbehandlung   | Custom Exception Handler fangen <code>`BusinessLogicError`</code> ab und liefern sauberes JSON.   | Verhindert den Absturz der Anwendung und schützt interne System-Stacktraces vor Endusern.          |

## 4. Technologie-Stack & Komponenten

| Schicht           | Technologie / Bibliothek                   | Begründung & Einsatzzweck                                                                                 |
|-------------------|--------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Backend Framework | Python 3.11 / Flask                        | Maximale Kompatibilität mit bestehenden E-Mail-Diensten. Leichtgewichtig und schnell im Docker-Container. |
| ORM & Datenbank   | PostgreSQL / Flask-SQLAlchemy              | ACID-Sicherheit für Belegungszeiten, Fremdschlüssel und flexible JSONB-Felder für Audit-Logs.             |
| Authentifizierung | Flask-JWT-Extended                         | Zustandlose Tokens im HTTP-Header zur Durchsetzung granularer API-Berechtigungen.                         |
| Frontend / UI     | Jinja2 / FullCalendar.js v6 / Tailwind CSS | Integrierte Ausführung im Flask-Container. Vollständig adaptiv für Desktop, Tablet und Smartphones.       |
| KI-Engine         | Ollama (Lokales LLM API)                   | 100 % DSGVO-konforme Verarbeitung im eigenen Netzwerk ohne externe API-Kosten.                            |
| Deployment        | Docker & Docker Compose                    | Isolierte Umgebung zur sauberen Trennung von Anwendungs- und Datenbank-Containern.                        |

## 5. Rollen- und Rechtekonzept (RBAC Matrix)

| Rolle              | Sichtbereich           | Schreibrechte       | Konflikt-Override    | Sonderrechte                                                          |
|--------------------|------------------------|---------------------|----------------------|-----------------------------------------------------------------------|
| CEO (Kai Bellmann) | Alle Benutzer (Global) | Uneingeschränkt     | Ja (mit Warnung)     | Unveränderbarer God-Mode; Vergibt TL-Rechte; Verwaltet Abwesenheiten. |
| Administrator      | Gesamtes Unternehmen   | Betriebliche Events | Nein (Hart gesperrt) | Benutzer-Onboarding; Rollen-Editor; Feiertage; Hard Delete.           |
| Team Leader (TL)   | Zugeordnetes Team      | Nur eigenes Team    | Nein (Hart gesperrt) | Einsicht in fremde Teams nur bei expliziter Freigabe durch CEO.       |
| Trainer            | Nur eigener Kalender   | Keine (Leserecht)   | Entfällt             | Einladungen annehmen/ablehnen (Pflichtbegründung bei Ablehnung).      |

## 6. Detaillierte Modulspezifikationen

### Modul A: Ansichten & KI-Vergleichs-Engine (Casino Layout)

- **Standard-Ansichten:** Tag, Woche, Monat, Jahr und Agenda-Liste mit Umschalter.
- **Nebeneinander-Vergleich (Casino Card Layout):** Ausgewählte Kalender werden in parallelen Karten/Spalten nebeneinander dargestellt. Überlagerungen werden ausgeschlossen.
- **Linkes Panel:** Zeigt eine KI-Zusammenfassung der Konflikte sowie gemeinsame freie Zeitfenster (Free Slots).
- **Zweistufige KI-Analyse:** 1. Deterministischer Python-Scan der Zeitstempel. 2. Generierung eines Textberichts durch Ollama.

### Modul B: Termin-Management & Pufferregeln

- **Pufferzeiten (Buffer Time):** Automatische Sperre von 15–30 Minuten vor/nach Terminen für TLs. *Ausnahme: CEO Kai Bellmann kann Pufferzeiten beliebig übersteuern.*
- **Dynamische Skill-Tags:** Beliebiges Anlegen von Qualifikations-Tags. Warnung bei Zuweisung an unqualifizierte Trainer.
- **Audit-Log pro Termin:** Historien-Fenster an jedem Event zur Rückverfolgung aller Änderungen.

### Modul C: Kollisions-Sperren & RSVP-Workflow

- **Doppelbuchungsschutz:** TLs erhalten bei Überschneidungen ein hartes Rebound mit Angabe des Erstbuchers.
- **CEO-Override-Dialog:** Bucht Kai Bellmann einen belegten Trainer, erscheint ein Bestätigungs-Popup.
- **RSVP mit Pflichtbegründung:** Bei Ablehnung durch den Trainer MUSS ein Grund eingegeben werden.
- **Aufgaben-Verlagerung:** Abgelehnte Termine verschwinden sofort beim Trainer und wandern als Aufgabe zum TL.

### Modul D: Suche, Benachrichtigungen & Lifecycle

- **Omni-Channel-Benachrichtigung:** In-App-Glockensymbol plus automatischer E-Mail-Versand.
- **Serientermine:** Unterstützung von Wiederholungen mit gezielter Übersteuerung von Einzelkonflikten.
- **Soft & Hard Delete:** Standardmäßiges Soft-Delete zur Erhaltung der Historie. Hard-Delete nur für Admins/CEO.
- **Rechteentzug-Report:** Bei Entzug von TL-Rechten erzeugt das System einen Bereinigungs-Report für Kai.

## 7. Datenbank-Schema (PostgreSQL Entitäten)

| Tabelle                  | Schlüssel-Felder                                                       | Beschreibung & Zweck                                                              |
|--------------------------|------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| `users`                  | `id, email, password_hash, role_id`                                    | Benutzerkonto mit eindeutiger Firmen-E-Mail und Zuordnung der Sicherheitsrolle.   |
| `roles`                  | `id, name, permissions_json, is_custom`                                | Rollenmatrix. Vordefinierte Rollen (CEO, Admin) und benutzerdefinierte Rollen.    |
| `events`                 | `id, title, start_time, end_time, created_by, assigned_to, is_deleted` | Zentrales Event-Register inklusive Soft-Delete-Flag und Pufferzeit-Parametern.    |
| `event_rsvps`            | `id, event_id, user_id, status, rejection_reason`                      | Speichert den Rückmeldestatus (ACCEPTED, REJECTED) samt Pflichtbegründung.        |
| `audit_logs`             | `id, event_id, user_id, action, timestamp, details_json`               | Unveränderbare Historie aller Systemaktionen zur lückenlosen Nachvollziehbarkeit. |
| `skills` & `user_skills` | `id, name` / `user_id, skill_id`                                       | Verwaltung dynamischer Qualifikationstags und Zuordnung zu Trainern.              |

## 8. Freigabe & Projekt-Start

Dieses Pflichtenheft dient als verbindliche Grundlage für die Entwicklung des internen Kalendersystems von Bellmann Engineering. Nach Freigabe beginnt Phase 1 der Implementierung.

**Kai Bellmann**  
CEO, Bellmann Engineering

**Entwicklungsteam**  
Lead Software Engineer